Class 16: Modeling, Experiment Tracking, Evaluation, and Decision-Making

Data-driven Systems Engineering

Agenda

- Modeling as the choice of a hypothesis class and a loss function.
- Matching model families to problem types and operational constraints.
- Experimentation as an engineering discipline, and what must be tracked to compare models fairly.
- Hyperparameter search strategies and experiment logging with Neptune.
- From a tuned run to a traceable artifact.
- Evaluation metrics that match the task.
- From a prediction to a decision: thresholds, uncertainty, and operational cost.
- Decision-support interfaces and the conditions under which trust breaks down.

What is modeling?

- A model is a function $f : \mathcal{X} \to \mathcal{Y}$ chosen from a hypothesis class $\mathcal{H}$ (e.g. linear functions, decision trees) so that it generalizes to unseen samples.
- Learning selects f by minimizing the empirical risk on the training data,

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i)),$$

  where the loss L quantifies the cost of an error; the quantity of interest is the expected risk on new data.
- The choice of $\mathcal{H}$ encodes assumptions about structure, noise, and decision boundaries, and determines the trade-off among accuracy, interpretability, computational cost, and maintainability.

Questions that precede the choice of a model

1. Which decision or prediction does the model support?
2. What is the target space: a class, a score, or a continuous value (cf. Class 13)?
3. Which errors are costly, and how costly relative to each other?
4. What are the latency constraints for training and inference?
5. How frequently is the data distribution expected to change?

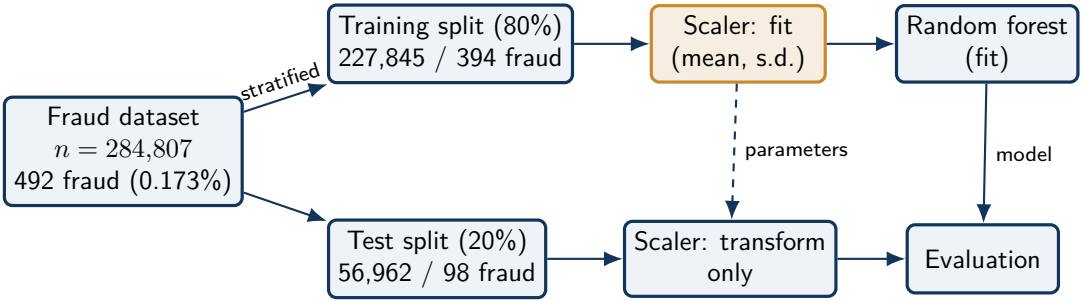
Model families in context

Linear models	Computationally efficient, interpretable, and strong baselines.
Tree-based models	Suited to heterogeneous tabular data and non-linear interactions.
Probabilistic models	Appropriate when uncertainty and conditional structure must be modeled explicitly.
Neural networks	Suited to large unstructured datasets and highly non-linear problems.
Online models	Updated incrementally as new observations arrive (Class 18).

Examples from the course

- **Fraud detection:** random forest or gradient boosting, followed by the choice of a decision threshold.
- **Wine quality regression:** gradient boosting tuned by grid search and tracked in Neptune (`Class7_Model_Tuning_Neptune.ipynb`).
- **Pasteurization forecasting:** incremental regression, treated in Class 18.
- **Industrial anomaly detection:** useful only if the cost of an alert and the acceptable false-positive rate are defined in advance.

Repository example: a baseline training pipeline



- From `streamlit/modeling.py`; `notebooks/Class6_Model_Training_.ipynb` applies the same pattern to the Iris dataset.
- **Stratification** keeps the fraud rate at 0.173% in both splits (394 and 98 positive samples); a random split could leave the test set with noticeably fewer or more positives.
- **The scaler is fitted on the training split only** and then applied to the test split with the same parameters, to avoid preprocessing leakage (Class 13).

Experimentation is an engineering discipline

- The purpose of experimentation is not to accumulate scores, but to support a credible model choice.
- Every run should answer a stated question: does a new feature, a different model family, or additional tuning justify its complexity?
- The evaluation protocol and the minimum improvement considered meaningful must be fixed before results are inspected.
- Without a stated question, an experiment tracker stores results without providing evidence.

What must be tracked, and why

Dataset version or extraction date	Allows a result to be reproduced on the same data.
Feature pipeline version	Separates the effect of preprocessing changes from that of model changes.
Hyperparameters and random seeds	Identify the exact configuration that produced a result.
Metrics, confusion matrices, artifacts	Allow the trained model, not only a number, to be audited.
Code commit or notebook snapshot	Allows the logic behind a run to be replayed later (Class 17).

Fair comparison and sources of bias

A fair comparison uses

- the same data split or validation protocol;
- the same preprocessing, unless preprocessing is the object of the experiment;
- metrics chosen for the actual decision;
- a measure of variability, e.g. mean $\pm$ standard deviation across cross-validation folds.

Bias is introduced by

- reusing the test set until it becomes part of tuning (model-selection leakage);
- hidden notebook state and untracked edits;
- reporting only the best run.

Hyperparameter search strategies

Manual tuning	Small problems; useful to build intuition.
Grid search	Exhaustive evaluation of a small, structured search space.
Random search	Better coverage when only a few of many hyperparameters matter (Bergstra and Bengio, 2012).
Bayesian optimization	A surrogate model of the score guides the search; suited to expensive models.

- A tuned model should outperform a simple, default, or previously deployed baseline by more than the variability of the estimate.

Neptune example: search space and protocol

Search space of a gradient boosting regressor:

Hyperparameter	Values
`n_estimators`	100, 200, 300
`max_depth`	3, 5, 7
`learning_rate`	0.05, 0.1, 0.2
`min_samples_split`	2, 5, 10

Protocol:

- Data: red wine quality, 1,599 samples; target `quality`.
- Split: 75% training, 25% test (fixed seed).
- Grid search: $3^4 = 81$ configurations $\times$ 5-fold cross-validation $=$ 405 fits, scored by R^2 on the training split.
- The test split is used once, after selection.

From `notebooks/Class7_Model_Tuning_Neptune.ipynb`, which combines `GridSearchCV` with the Neptune experiment tracker.

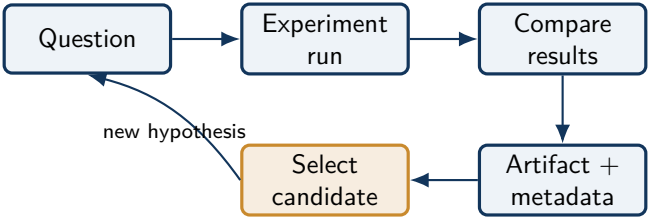
Neptune example: a tracked run record

Every run stores its configuration, results, and artifacts together, in the same place, automatically:

Field	Value logged by the notebook
`best/parameters`	learning_rate $= 0.05$, max_depth $= 7$, min_samples_split $= 5$, n_estimators $= 200$
`best/cv_score`	mean R^2 over the 5 folds of the selected configuration
`test/mse`	0.349
`test/r2`	0.436
`artifacts/model`	`best_gb_model.pkl` (serialized estimator)

- Mean squared error (MSE) and coefficient of determination (R^2) are computed once on the test split, after selection.
- The pattern is tool-independent: MLflow, Weights & Biases, or an internal registry store the same record. The serialized artifact is the bridge to deployment (Class 20).

From a notebook run to a governed asset



Common tuning mistakes

- Tuning hyperparameters on the test set.
- Comparing runs trained with different preprocessing.
- Reporting only the best score, without its variability or computational cost.
- Ignoring that a slightly less accurate model may be considerably easier to deploy, explain, or monitor.

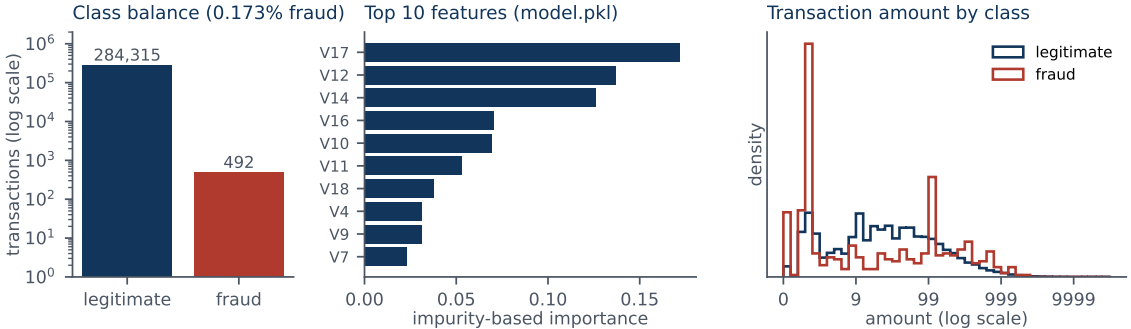
Evaluation metrics must match the task

- Tracking identifies which run performed best; it does not establish whether that run is adequate for the decision it supports.
- **Imbalanced classification:** with true and false positives and negatives (TP, FP, TN, FN),

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN},$$

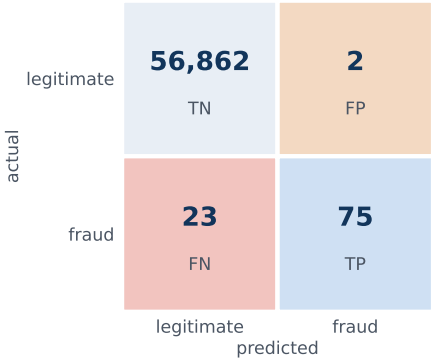
  summarized over all thresholds by the area under the precision–recall curve (PR-AUC); accuracy alone is misleading.
- **Regression:** mean absolute error (MAE) and root mean squared error (RMSE), expressed in the unit that matters operationally.
- **Calibration:** required whenever a downstream action depends on the predicted probability rather than only on the predicted class.

The fraud dataset at a glance



- **Imbalance:** 492 of 284,807 transactions (0.173%) are fraudulent; the first difficulty is the rarity of the positive class, not the choice of algorithm.
- **Importance:** three principal components (`V17`, `V12`, `V14`) account for 43% of the impurity-based importance of the deployed random forest; importance measures association, not causal effect.
- **Skewness:** amounts span four orders of magnitude, and fraudulent transactions tend to be smaller (median 9.25 versus 22.00); scaling, evaluation, and threshold choice interact with this shape.

Repository example: inspecting the errors



Test split (56,962 transactions), best random forest of the grid search in `Credit_Card_Fraud_Detection_Solution.ipynb`.

$$\text{precision} = \frac{75}{75+2} = 0.97, \qquad \text{recall} = \frac{75}{75+23} = 0.77$$

$$\text{accuracy} = \frac{56{,}862+75}{56{,}962} = 0.9996$$

- 23 of the 98 fraudulent transactions are missed (FN), while only 2 legitimate ones are flagged (FP).
- Accuracy, at 99.96%, conceals both numbers; the confusion matrix exposes them directly.
- Whether 23 missed frauds are acceptable depends on the cost of a missed fraud relative to a false alarm, which motivates the threshold analysis that follows.

Why accuracy fails under class imbalance

Fraud example

With 0.173% fraudulent transactions, a classifier that always predicts "legitimate" attains an accuracy of 99.83%, a recall of 0, and no operational value.

- Precision, recall, and the decision threshold are therefore part of the engineering design, not a post-hoc check.

A model creates value only through an interface

- A trained model that cannot be invoked or interpreted has limited practical value, regardless of its offline metrics.
- An interface makes predictions available to users, services, and operational workflows.
- The interface also determines trust: input validation, response format, and clarity matter as much as the metric used to select the model.

A prediction is not a decision

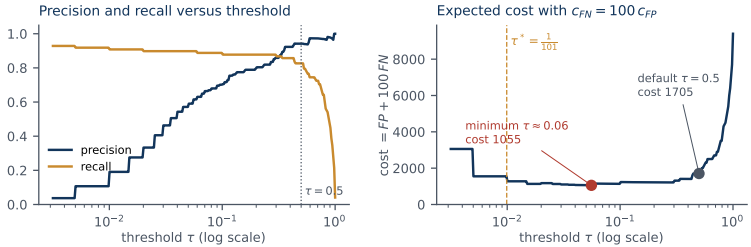
- A fraud score suggests escalation, not guilt; a process alert suggests investigation, not automatic shutdown.
- A classifier outputs an estimate $\hat{p}(\mathbf{x})$ of $P(y = 1 \mid \mathbf{x})$; the decision rule is $\hat{y} = \mathbf{1}[\hat{p}(\mathbf{x}) \geq \tau]$.
- If $\hat{p}$ is calibrated and c_{FP}, c_{FN} denote the costs of a false positive and a false negative, the expected cost is minimized by

$$\tau^* = \frac{c_{FP}}{c_{FP} + c_{FN}}.$$

Engineering implication

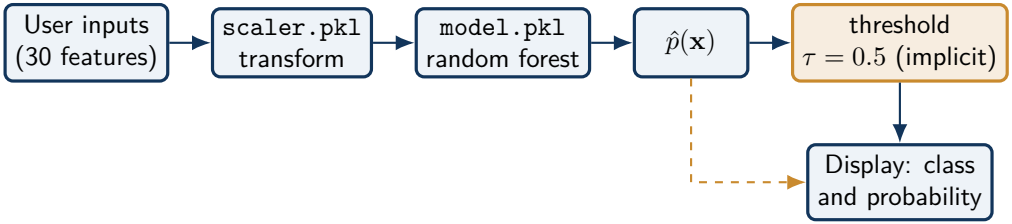
The threshold that turns a probability into a decision is a design choice derived from costs, not a by-product of the model (the default $\tau = 0.5$ assumes $c_{FP} = c_{FN}$).

Choosing the threshold: a cost analysis



- Deployed model (`streamlit/model.pkl`) on the test split, assuming a missed fraud costs 100 times a false alarm.
- **Default** $\tau = 0.5$: 81 frauds detected, 17 missed, 5 false alarms; cost 1,705. **Minimum** at $\tau \approx 0.06$: 88 detected, 10 missed, 55 false alarms; cost 1,055 (-38%).
- The theoretical $\tau^* = 1/101 \approx 0.01$ differs from the empirical minimum because random-forest scores (vote fractions) are not calibrated probabilities.
- In practice, τ must be chosen on a validation split; selecting it on the test split, as in this illustration, is model-selection leakage.

Repository example: decision support in Streamlit



- From `streamlit/pages/App_Credit_Card.py`: the interface shows both the predicted class and the estimated probability.
- The class comes from `model.predict`, i.e. the implicit threshold $\tau = 0.5$; under the cost assumption of the previous analysis, a threshold near 0.06 would detect 7 more frauds in the test split.
- The threshold should be an explicit, documented, and versioned parameter of the interface, not a default of the library.

Requirements for a decision-support dashboard

- An explicit statement of what the model does and does not do.
- Inputs that correspond to meaningful domain variables rather than technical internals.
- Outputs that support action: alert, score, trend, or explanation.
- References to data provenance, model version, and monitoring signals.

Where trust breaks down

- The interface conceals uncertainty behind a single, apparently precise number.
- The inputs presented to users do not match the assumptions under which the model was trained.
- The threshold was chosen once, informally, and never revisited as the data changed.
- The dashboard makes a demonstration model appear authoritative although it has not been validated for that purpose.

Repository connections

Repo activity

- `notebooks/Class6_Model_Training_.ipynb`: baseline split, tuning, evaluation, and serialization (Iris dataset).
- `notebooks/Class7_Model_Tuning_Neptune.ipynb`: grid search with Neptune tracking (wine quality regression).
- `Credit_Card_Fraud_Detection_Solution.ipynb`: evaluation of the fraud classifier.
- `streamlit/modeling.py` and `streamlit/pages/App_Credit_Card.py`: training, persistence of model and scaler, and the human-facing decision interface.

Papers and materials

- Bergstra, J., Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.
- Bergstra, J., Yamins, D., Cox, D. (2013). Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. *Proceedings of ICML*, PMLR 28(1), 115–123.
- Elkan, C. (2001). The foundations of cost-sensitive learning. *Proceedings of IJCAI*, 973–978.
- Saito, T., Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432.
- Niculescu-Mizil, A., Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of ICML*, 625–632.
- Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78–87.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly.
- scikit-learn User Guide: tuning hyperparameters; model evaluation.
- Tool documentation: Neptune; MLflow; Streamlit.

Take-away message

A model is selected by tracked, reproducible evidence,
evaluated with metrics that match the task,

and turned into a decision through a threshold
derived from the costs of its errors.

Thank you for your attention.

Next: Class 17 — version control with Git.